



```
//BFS
#include <bits/stdc++.h>
#define MAXLEN 1000009
using namespace std;

vector<vector<int>> Graph;
//遍历一个图（有向or无向都可），结果是将与起始点可达的所有点都变成black
int color[MAXLEN];
int d[MAXLEN],pi[MAXLEN];
int white=0,gray=1,black=2;
void BFS(vector<vector<int>> &G,int s){//G是图，可以用邻接表或者二维数组，start起始点，BFS一次只能获得
    int n=G.size();
    //白色未访问，灰色正在遍历，黑色遍历结束
    //d.assign(n,INT_MAX);//记录到原点s的距离
    //pi.assign(n,-1);//记录前面的节点，方便恢复路径
    color[s]=gray;
    d[s]=0;
    pi[s]=-1;
    queue<int> Q;
    Q.push(s);
    while(!Q.empty()){
        int u=Q.front();//front访问第一个
        Q.pop();//pop只删除第一个，无返回值
        for(auto v:G[u]){//遍历每个关联节点
            if(color[v]==white){
                color[v]=gray;
                d[v]=d[u]+1;
                pi[v]=u;
                Q.push(v);
            }
        }
        color[u]=black;
    }
}

//打印路径//G前面加&!!!
void print_Graph(vector<vector<int>> &G,int s,int v){
    if(v==s) {
        cout << s << " ";
    }
}
```

```

        return;
    }
    else if(pi[v]==-1){
        cout << "no road" << endl;
    }
    else{
        print_Graph(G,s,pi[v]);
        cout << v << " ";
    }
}

bool PRINT_PATH_RECURSIVE(int pi[], int s, int v) {
    if (v == s) {
        cout << s; // 到达源点，输出并返回true表示路径有效
        return true;
    }
    if (pi[v] == -1) {
        cout << "no path from " << s << " to " << v << " exists";
        return false; // 不可达，返回false终止递归
    }
    // 先递归打印前驱路径，若前驱路径无效则直接返回
    if (!PRINT_PATH_RECURSIVE(pi, s, pi[v])) {
        return false;
    }
    cout << " -> " << v; // 前驱路径有效时，输出当前节点
    return true;
}

int main(){
    int n,m;
    std::cin.tie(nullptr) -> sync_with_stdio(false);
    cin >> n >> m;
    Graph.resize(n);
    //d.resize(n);pi.resize(n);
    for(int i=0;i<m;i++){
        int start,end;
        cin >> start >> end;
        Graph[start-1].push_back(end-1);
        Graph[end-1].push_back(start-1);
    }
}

```

```

int cnt=0;
for(int i=0;i<n;i++){
    color[i]=white;
    d[i]=INT_MAX;
    pi[i]=-1;
} //使用前初始化!!!
for(int j=0;j<n;j++){
    if(color[j]==white){
        BFS(Graph,j);
        cout << j <<" : ";
        PRINT_PATH_RECURSIVE(pi,j,2); //打印改索引，有箭头
        cout << endl;
        cnt++;
    }
}
cout << cnt << endl;
return 0;
}

```

```

//DFS
#include <bits/stdc++.h>
#define MAXLEN 1000009
using namespace std;

//内部有初始化，得到d,f，G节点编号从0开始
vector<int> color;
vector<int> pi;//前驱节点
vector<int> d,f;//开始遍历u和结束遍历u，用于拓扑排序
int white=0,gray=1,black=2;
int cnt;//cnt可以记录有几个图
void DFS_visit(const vector<vector<int>> &G,int u,int &time){
    time++;
    d[u]=time;
    color[u]=gray;
    for(auto &v:G[u]){
        if(color[v]==white){
            pi[v]=u;
            DFS_visit(G,v,time);
        }
    }
    color[u]=black;
    time++;
    f[u]=time;
}

void DFS(const vector<vector<int>> &G){
    int size=G.size();
    if(size==0) return;
    color.assign(size,0);
    pi.assign(size,-1);
    d.assign(size,0);
    f.assign(size,0);
    int time=0;
    for(int i=0;i<size;i++){
        if(color[i]==white){
            cnt++;
            DFS_visit(G,i,time);
        }
    }
}

```

```

    }
}
//
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<vector<int>> G(n);
    for(int i = 0; i < m; i++){
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);    // 有向图
        // G[v].push_back(u); // 若是无向图
    }

    DFS(G);

    cout << "DFS tree count: " << cnt << "\n";
    for(int i = 0; i < n; i++){
        cout << i << ": d=" << d[i] << ", f=" << f[i] << "\n";
    }
}

```